

ACTIVIDAD 3

SQL & Web Security

Participante:

Deyra Salazar Avila

24 de octubre de 2026

EJERCICIO 01 | Nombre: [SELECT *]

Objetivo	Consultar todos los registros y columnas de una tabla mediante la instrucción SELECT.
Concepto SQL	**SELECT *** permite recuperar todas las columnas de una tabla y **FROM** indica de qué tabla se obtendrán los datos.
Consulta ejecutada	SELECT * FROM family_members
Resultado / captura	Con este commando se obtuvo 3 registros de la table family_members, con las columnas Id, name, gender, species y num_books_read. Tu resultado Esperado ✓ id name gender species num_books_read 1 Dave male human 200 2 Mary female human 180 3 Pickles male dog 0
Interpretación	La consulta recupera toda la información almacenada en la tabla, sin aplicar filtros ni restricciones sobre los registros o las columnas.
Aprendizaje	Aprendí como usar el comando SELECT* para visualizar el contenido de una tabla, ver su estructura y obtener los datos que tenga disponible.
Aplicación real	Se podría utilizar para consultar todos los registros de una tabla como un inventario de equipos tecnológicos con todas las características EOL, ubicación, etc. También se puede usar para obtener la información de una lista de clientes con nombre, id, correo y toda la información que tenga disponible esa lista.

EJERCICIO 02 | Nombre: [SELECT columnas específicas]

Objetivo	Consultar únicamente las columnas que se requiera ver de una tabla, evitando recuperar información innecesaria.
Concepto SQL	La instrucción SELECT permite especificar las columnas que se desean consultar, separándolas mediante comas. FROM identifica la tabla de origen.
Consulta ejecutada	SELECT name, species FROM family_members;
Resultado / captura	La consulta arrojó tres registros con únicamente las columnas name y species, mostrando los nombres Dave, Mary y Pickles, junto con sus respectivas especies. Tu resultado Esperado ✓ name species Dave human Mary human Pickles dog
Interpretación	La consulta selecciona exclusivamente los campos de nombre y especie de la tabla family_members, omitiendo las demás columnas y manteniendo todos los registros.
Aprendizaje	Aprendí a seleccionar columnas específicas mediante SQL, comprendiendo la importancia de recuperar únicamente los datos necesarios para una consulta. Esto permite optimizar el rendimiento de la base de datos, reducir el consumo de recursos y evitar la exposición innecesaria de información sensible, contribuyendo a una gestión más eficiente y segura de los datos.
Aplicación real	Se podría utilizar para consultar únicamente los datos necesarios de una tabla, como el nombre, dirección IP y estado de los equipos tecnológicos de una empresa, sin recuperar todas sus características. También se podría obtener información específica de una lista de clientes, como nombre y correo electrónico, sin acceder a otros datos personales. En ciberseguridad, se puede emplear para consultar únicamente la fecha, el usuario y el tipo de evento registrado en una tabla de auditoría, facilitando el análisis de información y reduciendo la exposición innecesaria de datos sensibles.

EJERCICIO 03 | Nombre: [WHERE ... Equals]

Objetivo	Aprender a filtrar los registros de una tabla mediante una condición de igualdad, obteniendo únicamente aquellos que cumplan con un criterio específico.
Concepto SQL	La cláusula WHERE permite filtrar registros según una condición. El operador = compara valores y devuelve únicamente las filas que coinciden exactamente con el valor indicado.
Consulta ejecutada	SELECT * FROM family_members WHERE species = 'dog';
Resultado / captura	La consulta devolvió un único registro correspondiente a Pickles, cuyo identificador es 3, género masculino, especie dog y cantidad de libros leídos igual a 0. Tu resultado Esperado ✓ id name gender species num_books_read 3 Pickles male dog 0
Interpretación	La consulta recupera todas las columnas de la tabla family_members, pero únicamente muestra los registros donde el campo especies tiene el valor dog, excluyendo aquellos que no cumplen la condición.
Aprendizaje	Aprendí a utilizar la cláusula WHERE junto con el operador de igual = para filtrar registros según un valor específico. Esto es importante porque permite realizar búsquedas más precisas, reducir la cantidad de información recuperada y facilitar el análisis de datos, optimizando las consultas y evitando revisar registros innecesarios.
Aplicación real	En una empresa, esta instrucción puede utilizarse para consultar únicamente los equipos tecnológicos cuyo estado sea «Activo» o que pertenezcan a una sede específica. También podría utilizarse para filtrar clientes según su categoría o trabajadores pertenecientes a un determinado departamento. En ciberseguridad, puede emplearse para identificar eventos asociados a un usuario, una dirección IP o un tipo específico de alerta, facilitando las investigaciones y auditorías de seguridad.

EJERCICIO 04 | Nombre: [WHERE ... Mayor que]

Objetivo	Aprender a filtrar registros de una tabla utilizando una condición numérica para recuperar únicamente aquellos cuyos valores sean mayores que un número determinado.
Concepto SQL	La cláusula WHERE permite establecer condiciones para filtrar registros. El operador de comparación > (mayor que) selecciona únicamente las filas cuyo valor numérico supera el límite especificado, excluyendo los valores iguales o inferiores.
Consulta ejecutada	SELECT * FROM family_members WHERE num_books_read > 190;
Resultado / captura	La consulta devolvió un único registro correspondiente a Dave, con identificador 1, género masculino, especie human y 200 libros leídos, ya que es el único miembro que supera los 190 libros. Tu resultado Esperado ✓ id name gender species num_books_read 1 Dave male human 200
Interpretación	La consulta recupera todas las columnas de la tabla family_members, pero únicamente muestra los registros cuyo campo num_books_read contiene un valor superior a 190. Los registros con valores iguales o inferiores quedan excluidos.
Aprendizaje	Aprendí a utilizar el operador > junto con la cláusula WHERE para realizar comparaciones numéricas y filtrar registros que superen un valor establecido. Esto es importante porque permite identificar información que excede determinados umbrales, optimizar la búsqueda de datos y facilitar el análisis de indicadores, especialmente en procesos de monitoreo, gestión y toma de decisiones.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar equipos tecnológicos cuya antigüedad supere los cinco años y evaluar su renovación. También permite consultar productos cuyo stock sea mayor a una cantidad determinada o identificar transacciones bancarias que superen un importe establecido. En ciberseguridad, puede emplearse para detectar direcciones IP con más de 100 intentos fallidos de autenticación o servidores cuyo consumo de CPU supere el 90 %, facilitando la identificación de situaciones que requieren atención.

EJERCICIO 05 | Nombre: [WHERE ... Mayor o igual que]

Objetivo	Aprender a filtrar registros de una tabla mediante una condición numérica que permita recuperar valores mayores o iguales a un límite establecido.
Concepto SQL	La cláusula WHERE permite filtrar registros utilizando condiciones. El operador ≥ (mayor o igual que) selecciona las filas cuyos valores sean superiores o iguales al número especificado, incluyendo el valor límite.
Consulta ejecutada	SELECT* FROM family_members WHERE num_books_read≥180;
Resultado / captura	La consulta devolvió dos registros: Dave, con 200 libros leídos, y Mary, con 180 libros leídos. Ambos cumplen la condición de haber leído al menos 180 libros. Tu resultado Esperado ✓ id name gender species num_books_read 1 Dave male human 200 2 Mary female human 180
Interpretación	La consulta recupera todas las columnas de la tabla family_members, pero únicamente muestra los registros cuyo campo num_books_read contiene un valor mayor o igual a 180. A diferencia del operador >, esta condición también incluye los registros que coinciden exactamente con el límite establecido.
Aprendizaje	Aprendí a utilizar el operador ≥ junto con la cláusula WHERE para filtrar registros considerando tanto los valores superiores como aquellos que coinciden con un límite determinado. Esto es importante porque permite establecer criterios mínimos de búsqueda, evaluar el cumplimiento de requisitos y analizar indicadores de manera precisa, evitando excluir registros que cumplen exactamente con el umbral definido.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar equipos tecnológicos con una antigüedad mayor o igual a un número de años y evaluar su renovación. También permite consultar clientes cuyas compras acumuladas alcancen o superen un importe determinado, o trabajadores con un numero de años o más de antigüedad. En ciberseguridad, puede emplearse para identificar servidores cuyo consumo de CPU sea igual o superior al 90 %, o usuarios con 3 o más intentos fallidos de autenticación, por ejemplo, facilitando el monitoreo y la detección de posibles riesgos.

EJERCICIO 06 | Nombre: [AND]

Objetivo	Aprender a combinar múltiples condiciones de búsqueda mediante el operador lógico AND, recuperando únicamente los registros que cumplan simultáneamente todos los criterios establecidos.
Concepto SQL	El operador lógico AND permite combinar dos o más condiciones dentro de la cláusula WHERE. Para que un registro sea seleccionado, todas las condiciones deben cumplirse.
Consulta ejecutada	SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';
Resultado / captura	La consulta devolvió dos registros correspondientes a Odie, con identificador 5 y altura de 40 cm, y Jumpy, con identificador 6 y altura de 35 cm. Ambos pertenecen a la especie dog y cumplen la condición de medir menos de 45 cm. Tu resultado Esperado ✓ id name gender species height_cm 5 Odie male dog 40 6 Jumpy male dog 35
Interpretación	La consulta recupera todas las columnas de la tabla friends_of_pickles, pero únicamente muestra los registros que cumplen simultáneamente dos condiciones: pertenecer a la especie dog y tener una altura inferior a 45 cm. Los registros que incumplen cualquiera de estas condiciones quedan excluidos.
Aprendizaje	Aprendí a utilizar el operador lógico AND para combinar múltiples condiciones dentro de una consulta SQL, comprendiendo que todas deben cumplirse para obtener un resultado. Esto es importante porque permite realizar búsquedas más precisas, reducir los registros innecesarios y aplicar criterios de selección más específicos, facilitando el análisis de información y la toma de decisiones basadas en diferentes atributos.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar equipos tecnológicos que tengan más de cinco años de antigüedad y se encuentren en estado activo, facilitando la planificación de su renovación. También, en ciberseguridad, puede emplearse para identificar eventos de autenticación fallida provenientes de una dirección IP específica, o alertas de severidad alta asociadas a un servidor determinado, facilitando el análisis y la investigación de incidentes.

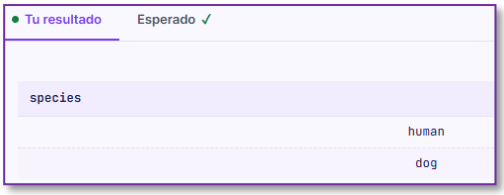
EJERCICIO 07 | Nombre: [OR]

Objetivo	Aprender a combinar múltiples condiciones de búsqueda mediante el operador lógico OR, recuperando los registros que cumplan al menos una de las condiciones establecidas.
Concepto SQL	El operador lógico OR permite combinar dos o más condiciones dentro de la cláusula WHERE. A diferencia de AND, no requiere que todas las condiciones se cumplan simultáneamente, sino que basta con que una sea verdadera para incluir el registro en el resultado.
Consulta ejecutada	SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';
Resultado / captura	La consulta devolvió cinco registros: Fry (30 cm), Leela (25 cm), Odie (40 cm), Jumpy (35 cm) y Sneakers (55 cm). Los cuatro primeros cumplen la condición de medir menos de 45 cm, mientras que Sneakers se incluye por pertenecer a la especie dog. ● Tu resultado Esperado ✓ id name gender species height_cm 3 Fry male cat 30 4 Leela female cat 25 5 Odie male dog 40 6 Jumpy male dog 35 7 Sneakers male dog 55
Interpretación	La consulta recupera todas las columnas de la tabla friends_of_pickles y selecciona los registros que cumplen al menos una de las dos condiciones: tener una altura inferior a 45 cm o pertenecer a la especie dog. Los registros que cumplen ambas condiciones aparecen una sola vez, mientras que aquellos que no cumplen ninguna quedan excluidos.
Aprendizaje	Aprendí a utilizar el operador lógico OR para combinar diferentes condiciones de búsqueda, comprendiendo que basta con cumplir una de ellas para recuperar un registro. Esto es importante porque permite realizar consultas más flexibles, ampliar los criterios de selección y analizar información que puede cumplir diferentes características, facilitando la identificación de registros relevantes sin necesidad de ejecutar múltiples consultas independientes.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar equipos tecnológicos que tengan más de cinco años de antigüedad o cuyo estado sea “Obsoleto” o “Backlevel”, facilitando la planificación de su renovación. En ciberseguridad, puede emplearse para identificar eventos de autenticación fallida o accesos provenientes de direcciones IP sospechosas, así como alertas de seguridad clasificadas con severidad alta o crítica, facilitando el monitoreo y la investigación de posibles amenazas.

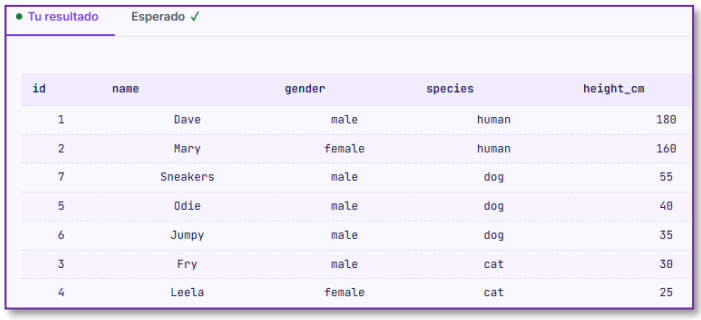
EJERCICIO 08 | Nombre: [IN]

Objetivo	Aprender a filtrar registros mediante una lista de valores específicos, utilizando los operadores IN y NOT IN para incluir o excluir información según determinados criterios.
Concepto SQL	El operador IN permite seleccionar registros cuyos valores coincidan con alguno de los elementos de una lista definida dentro de la cláusula WHERE. Por otro lado, NOT IN permite excluir los registros que coincidan con los valores indicados, recuperando únicamente aquellos que no pertenecen a dicha lista.
Consulta ejecutada	SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');
Resultado / captura	La consulta devolvió dos registros: Dave, con identificador 1, género masculino y altura de 180 cm, y Mary, con identificador 2, género femenino y altura de 160 cm. Ambos pertenecen a la especie human, por lo que no coinciden con los valores excluidos (cat y dog). ● Tu resultado Esperado ✓ id name gender species height_cm 1 Dave male human 180 2 Mary female human 160
Interpretación	La consulta arroja todas las columnas de la tabla friends_of_pickles, pero excluye los registros cuya especie sea cat o dog. Como resultado, únicamente se muestran los dos registros pertenecientes a la especie human, demostrando cómo NOT IN permite aplicar múltiples exclusiones mediante una sola condición.
Aprendizaje	Aprendí a utilizar los operadores IN y NOT IN para filtrar registros a partir de una lista de valores, comprendiendo que permiten incluir o excluir múltiples criterios dentro de una misma consulta. Esto es importante porque simplifica las condiciones de búsqueda, evita utilizar repetidamente operadores OR o AND, mejora la legibilidad del código SQL y facilita el análisis de información según diferentes categorías o características.
Aplicación real	En una empresa, el operador IN puede utilizarse para consultar equipos tecnológicos pertenecientes a determinadas sedes o departamentos, mientras que NOT IN permite excluir aquellos que se encuentren dados de baja u obsoletos.

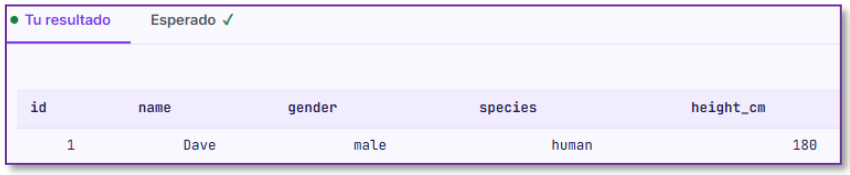
EJERCICIO 09 | Nombre: [DISTINCT]

Objetivo	Aprender a utilizar la cláusula DISTINCT para recuperar valores únicos de una columna, eliminando los resultados duplicados y aplicando condiciones específicas de búsqueda.
Concepto SQL	La palabra clave DISTINCT, utilizada después de SELECT, permite obtener valores únicos en los resultados de una consulta, evitando registros duplicados. Puede combinarse con WHERE para aplicar filtros y, cuando se seleccionan varias columnas, elimina las combinaciones repetidas de sus valores.
Consulta ejecutada	SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;
Resultado / captura	La consulta devolvió dos valores únicos en la columna species: human y dog. Aunque existen dos registros de humanos con alturas superiores a 50 cm, la instrucción DISTINCT evita repetir el valor human en el resultado. 
Interpretación	La consulta selecciona las especies de los registros de la tabla friends_of_pickles cuya altura supera los 50 cm. Posteriormente, DISTINCT elimina los valores repetidos, mostrando únicamente las especies diferentes que cumplen la condición, sin importar cuántos registros pertenezcan a cada una.
Aprendizaje	Aprendí a utilizar DISTINCT para obtener valores únicos y eliminar duplicados en los resultados de una consulta SQL, combinándolo con la cláusula WHERE para aplicar filtros específicos. Esto es importante porque permite identificar categorías diferentes dentro de una base de datos, simplificar la presentación de información, evitar resultados repetitivos y facilitar el análisis de datos sin modificar los registros originales almacenados.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar las diferentes marcas o modelos de equipos tecnológicos registrados en un inventario, evitando que aparezcan repetidos. También permite obtener una lista de ciudades donde residen los clientes o de departamentos a los que pertenecen los trabajadores. En ciberseguridad, puede emplearse para identificar las direcciones IP únicas que han generado eventos de seguridad, los usuarios que registraron intentos fallidos de autenticación o los distintos tipos de amenazas detectadas, facilitando el análisis y la elaboración de reportes.

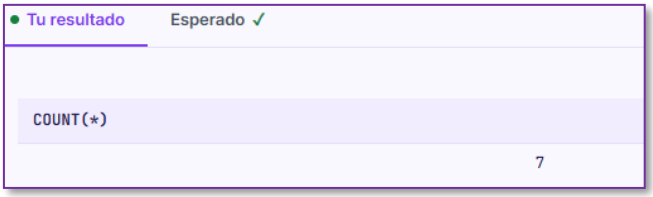
EJERCICIO 10 | Nombre: [ORDER BY]

Objetivo	Aprender a ordenar los registros de una tabla mediante la cláusula ORDER BY, utilizando un criterio numérico para presentar los resultados en orden descendente.
Concepto SQL	La cláusula ORDER BY permite organizar los resultados de una consulta según los valores de una o varias columnas. Puede utilizarse ASC para ordenar de forma ascendente (menor a mayor o alfabéticamente de A a Z) y DESC para ordenar de forma descendente (mayor a menor o de Z a A). Si no se especifica un orden, se utiliza ASC de manera predeterminada.
Consulta ejecutada	SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;
Resultado / captura	La consulta devolvió los siete registros de la tabla friends_of_pickles, ordenados de mayor a menor altura: Dave (180 cm), Mary (160 cm), Sneakers (55 cm), Odie (40 cm), Jumpy (35 cm), Fry (30 cm) y Leela (25 cm). 
Interpretación	La consulta recupera todas las columnas de la tabla friends_of_pickles y organiza los registros según el campo height_cm, utilizando DESC para mostrar primero los valores más altos y finalmente los más bajos. Esta operación modifica únicamente el orden de presentación de los resultados, sin alterar los datos almacenados
Aprendizaje	Aprendí a utilizar la cláusula ORDER BY para organizar los resultados de una consulta SQL en orden ascendente o descendente, según los valores de una columna. Esto es importante porque permite presentar la información de manera estructurada, identificar rápidamente los valores más altos o bajos, establecer prioridades y facilitar el análisis comparativo de datos, contribuyendo a una mejor interpretación de los resultados y a la toma de decisiones.
Aplicación real	En una empresa, esta instrucción puede utilizarse para ordenar los equipos tecnológicos según su antigüedad, mostrando primero aquellos que requieren una renovación prioritaria. También permite organizar las transacciones bancarias por importe, de mayor a menor, o clasificar productos según sus niveles de inventario. En ciberseguridad, puede emplearse para ordenar los eventos de seguridad por fecha, mostrando los más recientes, o clasificar las direcciones IP según la cantidad de intentos fallidos de autenticación, facilitando la identificación de actividades que requieren mayor atención.

EJERCICIO 11 | Nombre: [LIMIT # de filas devueltas]

Objetivo	Aprender a limitar la cantidad de registros devueltos por una consulta SQL mediante la cláusula LIMIT, combinándola con ORDER BY para obtener resultados específicos según un criterio de ordenamiento.
Concepto SQL	La cláusula LIMIT permite establecer el número máximo de filas que devolverá una consulta. Al combinarse con ORDER BY, permite seleccionar los primeros registros según un orden determinado, ya sea ascendente (ASC) o descendente (DESC), facilitando la recuperación de información relevante sin mostrar todos los registros de una tabla.
Consulta ejecutada	SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;
Resultado / captura	La consulta devolvió un único registro correspondiente a Dave, con identificador 1, género masculino, especie human y altura de 180 cm. Este resultado corresponde al registro con la mayor altura de la tabla friends_of_pickles. 
Interpretación	La consulta recupera todas las columnas de la tabla friends_of_pickles, ordena los registros de mayor a menor según el campo height_cm mediante ORDER BY DESC y utiliza LIMIT 1 para mostrar únicamente el primer registro, correspondiente al miembro de mayor altura.
Aprendizaje	Aprendí a utilizar la cláusula LIMIT para controlar la cantidad de registros devueltos por una consulta SQL y combinarla con ORDER BY para seleccionar los resultados más relevantes según un criterio establecido. Esto es importante porque permite reducir la cantidad de información presentada y transferida, facilitar la identificación de valores máximos o mínimos y mejorar la eficiencia de las consultas, especialmente cuando se trabaja con grandes volúmenes de datos.
Aplicación real	En una empresa, esta instrucción puede utilizarse para identificar los diez equipos tecnológicos con mayor antigüedad y priorizar su renovación. También permite consultar los cinco clientes con mayores compras o las diez transacciones bancarias de mayor importe para su análisis. En ciberseguridad, puede emplearse para obtener las diez direcciones IP con mayor cantidad de intentos fallidos de autenticación, o consultar los eventos de seguridad más recientes, facilitando la priorización de investigaciones y actividades de monitoreo.

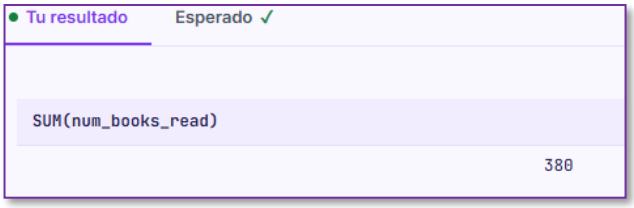
EJERCICIO 12 | Nombre: [COUNT(*)]

Objetivo	Aprender a utilizar la función COUNT(*) para determinar la cantidad total de registros almacenados en una tabla, sin necesidad de visualizar individualmente cada fila.
Concepto SQL	La función de agregación COUNT(*) permite contar todas las filas de una tabla, independientemente de los valores que contengan sus columnas. Puede combinarse con la cláusula WHERE para contabilizar únicamente los registros que cumplan determinadas condiciones, facilitando la obtención de indicadores cuantitativos.
Consulta ejecutada	SELECT COUNT(*) FROM friends_of_pickles;
Resultado / captura	La consulta devolvió el valor 7, correspondiente a la cantidad total de registros almacenados en la tabla friends_of_pickles. 
Interpretación	La consulta utiliza la función COUNT(*) para contabilizar todas las filas de la tabla friends_of_pickles. En lugar de mostrar la información individual de cada registro, devuelve un único resultado numérico que representa la cantidad total de filas existentes.
Aprendizaje	Aprendí a utilizar la función COUNT(*) para contabilizar los registros almacenados en una tabla SQL sin necesidad de recuperar toda su información. Esto es importante porque permite obtener indicadores cuantitativos, verificar la cantidad de registros disponibles y analizar grandes volúmenes de datos de manera más eficiente. Además, al combinarla con condiciones de búsqueda, facilita la elaboración de estadísticas, reportes y actividades de monitoreo.
Aplicación real	En una empresa, esta función puede utilizarse para conocer la cantidad total de equipos tecnológicos registrados en un inventario o contabilizar los trabajadores de una organización. También permite determinar cuántos clientes se encuentran registrados o cuántas transacciones bancarias se realizaron durante un periodo determinado. En ciberseguridad, puede emplearse para contar los intentos fallidos de autenticación, los incidentes registrados o las alertas de seguridad generadas por un sistema, facilitando el seguimiento de indicadores y la identificación de posibles anomalías.

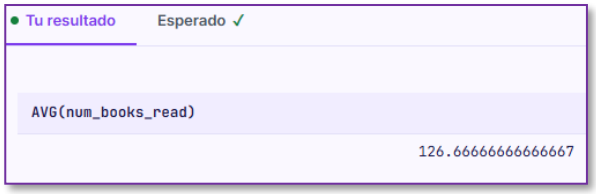
EJERCICIO 13 | Nombre: [COUNT(*) ... WHERE]

Objetivo	Aprender a combinar la función COUNT(*) con la cláusula WHERE para contabilizar únicamente los registros que cumplan una condición específica dentro de una tabla.
Concepto SQL	La función de agregación COUNT(*) permite contar registros, mientras que la cláusula WHERE establece condiciones para filtrar las filas antes de realizar el conteo. Al combinarlas, se obtiene la cantidad de registros que cumplen un criterio determinado, sin necesidad de recuperar toda su información.
Consulta ejecutada	SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';
Resultado / captura	La consulta devolvió el valor 3, correspondiente a los tres registros de la especie dog almacenados en la tabla friends_of_pickles. 
Interpretación	La consulta utiliza la cláusula WHERE para seleccionar únicamente los registros cuyo campo species sea igual a dog. Posteriormente, la función COUNT(*) contabiliza las filas que cumplen dicha condición, devolviendo un único resultado numérico de 3, en lugar de mostrar los datos individuales.
Aprendizaje	Aprendí a combinar la función COUNT(*) con la cláusula WHERE para contabilizar registros que cumplen condiciones específicas dentro de una tabla. Esto es importante porque permite obtener estadísticas más precisas, analizar información por categorías y medir el cumplimiento de determinados criterios. Además, facilita la elaboración de indicadores de gestión y reportes sin necesidad de recuperar o revisar individualmente todos los registros.
Aplicación real	En un hospital, esta consulta puede utilizarse para contabilizar cuántos pacientes se encuentran hospitalizados en una determinada especialidad, facilitando la distribución de camas y recursos médicos. En una empresa de logística, puede ayudar a conocer cuántos pedidos permanecen pendientes de entrega y planificar las operaciones de distribución. En una universidad, puede emplearse para determinar cuántos estudiantes aprobaron una asignatura específica, facilitando la elaboración de estadísticas académicas y la evaluación de resultados.

EJERCICIO 14 | Nombre: [SUM]

Objetivo	Aprender a utilizar la función SUM() para calcular la suma total de los valores numéricos almacenados en una columna de una tabla, obteniendo un resultado acumulado.
Concepto SQL	La función de agregación SUM() permite sumar los valores numéricos de una columna y devolver un resultado total. Puede combinarse con la cláusula WHERE para sumar únicamente los valores que cumplan determinadas condiciones.
Consulta ejecutada	SELECT SUM(num_books_read) FROM family_members;
Resultado / captura	La consulta devolvió el valor 380, correspondiente a la suma de los libros leídos por los tres miembros de la tabla family_members. 
Interpretación	La consulta utiliza la función SUM() sobre la columna num_books_read para sumar los valores registrados en las tres filas de la tabla. Como resultado, obtiene un total de 380 libros leídos, sin necesidad de realizar cálculos manuales ni mostrar individualmente cada registro.
Aprendizaje	Aprendí a utilizar la función de agregación SUM() para calcular el total acumulado de los valores numéricos almacenados en una columna. Esto es importante porque permite automatizar cálculos, reducir errores asociados al procesamiento manual y obtener indicadores cuantitativos de manera eficiente. Además, facilita la elaboración de reportes financieros, operativos y estadísticos que contribuyen al seguimiento de recursos, evaluación de resultados y toma de decisiones empresariales.
Aplicación real	En una empresa minera, esta función puede utilizarse para calcular la cantidad total de toneladas de mineral extraídas durante un periodo y evaluar el cumplimiento de las metas de producción. En una compañía eléctrica, puede ayudar a sumar el consumo energético registrado por diferentes instalaciones para conocer la demanda total. Asimismo, en el área financiera de una organización, puede emplearse para calcular el monto acumulado de gastos operativos o ingresos por ventas, facilitando el control presupuestario y la elaboración de reportes de gestión.

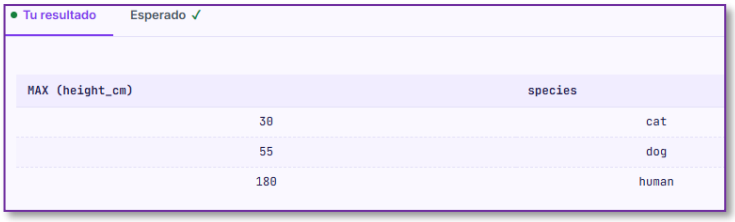
EJERCICIO 15 | Nombre: [AVG]

Objetivo	Aprender a utilizar la función AVG() para calcular el promedio de los valores numéricos almacenados en una columna de una tabla, obteniendo un indicador representativo del conjunto de datos.
Concepto SQL	La función de agregación AVG() permite calcular la media aritmética de los valores numéricos de una columna. Para ello, suma los valores y los divide entre la cantidad de registros con valores no nulos. Puede combinarse con WHERE para calcular promedios de registros que cumplan determinadas condiciones.
Consulta ejecutada	SELECT AVG(num_books_read) FROM family_members;
Resultado / captura	La consulta devolvió el valor 126.66666666666667, equivalente a un promedio aproximado de 126.67 libros leídos por integrante. 
Interpretación	La consulta utiliza la función AVG() sobre la columna num_books_read para calcular el promedio de libros leídos por los tres integrantes de la tabla family_members. El resultado se obtiene sumando los valores (200 + 180 + 0 = 380) y dividiéndolos entre los tres registros, obteniendo aproximadamente 126.67 libros por integrante.
Aprendizaje	Aprendí a utilizar la función de agregación AVG() para calcular el promedio de los valores numéricos almacenados en una tabla SQL. Esto es importante porque permite analizar tendencias generales, evaluar el comportamiento de indicadores y comparar resultados sin necesidad de realizar cálculos manuales. Además, facilita la identificación de desviaciones respecto a un valor promedio, contribuyendo a la elaboración de estadísticas, evaluación del desempeño y toma de decisiones basadas en datos.
Aplicación real	En una empresa de telecomunicaciones, esta función puede utilizarse para calcular el tiempo promedio de resolución de averías y evaluar la calidad del servicio técnico. En una entidad bancaria, permite determinar el tiempo promedio de atención de solicitudes de crédito, identificando oportunidades de mejora en los procesos. Asimismo, en una empresa de transporte, puede emplearse para calcular el consumo promedio de combustible por vehículo, facilitando la evaluación de la eficiencia operativa y la optimización de costos.

EJERCICIO 16 | Nombre: [MAX y MIN]

Objetivo	Aprender a utilizar las funciones de agregación MAX() y MIN() para identificar los valores más altos y más bajos de una columna, facilitando el análisis de los extremos dentro de un conjunto de datos.
Concepto SQL	Las funciones de agregación MAX() y MIN() permiten obtener el valor máximo y mínimo, respectivamente, de una columna. Pueden utilizarse sobre datos numéricos, fechas y otros tipos de datos comparables. También pueden combinarse con la cláusula WHERE para identificar valores extremos dentro de un conjunto específico de registros.
Consulta ejecutada	SELECT MAX(num_books_read) FROM family_members;
Resultado / captura	La consulta devolvió el valor 200, correspondiente a la mayor cantidad de libros leídos registrada en la tabla family_members. 
Interpretación	La consulta utiliza la función MAX() sobre la columna num_books_read para identificar el valor numérico más alto entre todos los registros de la tabla. Como resultado, devuelve 200, sin mostrar los demás valores ni identificar directamente el registro al que pertenece.
Aprendizaje	Aprendí a utilizar las funciones MAX() y MIN() para identificar los valores máximos y mínimos dentro de una tabla SQL. Esto es importante porque permite reconocer los extremos de un conjunto de datos, evaluar rangos de comportamiento e identificar posibles desviaciones o situaciones excepcionales. Además, facilita el análisis de indicadores, la comparación de resultados y la toma de decisiones basadas en valores críticos, sin necesidad de revisar individualmente todos los registros.
Aplicación real	En una empresa industrial, estas funciones pueden utilizarse para identificar la temperatura máxima y mínima registrada por los sensores de una maquinaria, facilitando el control de las condiciones operativas. En una compañía aérea, permiten determinar los tiempos máximos y mínimos de retraso de los vuelos para evaluar la puntualidad del servicio. Asimismo, en un centro de datos, pueden emplearse para identificar los valores extremos de temperatura ambiental o consumo eléctrico registrados durante un periodo, contribuyendo al monitoreo de la infraestructura y la prevención de fallas.

EJERCICIO 17 | Nombre: [GROUP BY]

Objetivo	Aprender a utilizar la cláusula GROUP BY para agrupar registros que comparten características comunes y aplicar funciones de agregación para obtener resultados específicos por cada grupo.
Concepto SQL	GROUP BY permite agrupar las filas de una tabla según los valores de una o varias columnas. Generalmente, se combina con funciones de agregación como COUNT(), SUM(), AVG(), MAX() y MIN() para realizar cálculos independientes sobre cada grupo, facilitando la elaboración de resúmenes estadísticos.
Consulta ejecutada	SELECT MAX (height_cm), species FROM friends_of_pickles GROUP BY species;
Resultado / captura	La consulta devolvió tres registros correspondientes a las especies existentes en la tabla friends_of_pickles, mostrando la altura máxima de cada una.  Tu resultado Esperado ✓ MAX (height_cm) species 30 cat 55 dog 180 human
Interpretación	La consulta agrupa los registros de la tabla friends_of_pickles según el campo species. Posteriormente, aplica la función MAX() sobre height_cm para identificar la mayor altura registrada dentro de cada grupo. Como resultado, devuelve una fila por especie, mostrando su respectiva altura máxima.
Aprendizaje	Aprendí a utilizar la cláusula GROUP BY para organizar registros según características comunes y aplicar funciones de agregación de manera independiente a cada grupo. Esto es importante porque permite transformar grandes volúmenes de datos en información resumida, comparar indicadores entre diferentes categorías e identificar patrones o diferencias relevantes. Además, facilita la elaboración de reportes estadísticos, el seguimiento de indicadores de gestión y la toma de decisiones basadas en información consolidada.
Aplicación real	En una cadena de supermercados, esta instrucción puede utilizarse para agrupar las ventas por sucursal y calcular el importe máximo de una venta registrada en cada establecimiento. En una empresa de seguros, permite agrupar los siniestros por tipo de cobertura y determinar el monto máximo de indemnización correspondiente a cada categoría. Asimismo, en una institución educativa, puede emplearse para agrupar las calificaciones por asignatura y obtener la nota máxima alcanzada en cada curso, facilitando la comparación de resultados académicos.

EJERCICIO 18 | Nombre: [Consultas anidadas]

Objetivo	Aprender a utilizar consultas anidadas para obtener información mediante una consulta SQL dentro de otra, utilizando el resultado de la consulta interna como condición para recuperar registros específicos.
Concepto SQL	Las consultas anidadas o subconsultas permiten incorporar una instrucción SELECT dentro de otra consulta SQL. La subconsulta puede calcular o recuperar valores que posteriormente son utilizados por la consulta principal para aplicar filtros o comparaciones. Pueden combinarse con funciones de agregación como MAX(), MIN() y AVG().
Consulta ejecutada	SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);
Resultado / captura	La consulta devolvió un único registro correspondiente a Dave, con identificador 1, especie human, dos piernas y 200 libros leídos. Este registro representa al integrante con la mayor cantidad de libros leídos dentro de la tabla family_members.  Tu resultado Esperado ✓ id name species num_books_read num_legs 1 Dave human 200 2
Interpretación	La subconsulta interna utiliza MAX(num_books_read) para determinar que la mayor cantidad de libros leídos es 200. Posteriormente, la consulta principal compara este resultado con los valores de la columna num_books_read mediante WHERE, recuperando todos los registros que coincidan con dicho máximo. En este caso, únicamente Dave cumple la condición.
Aprendizaje	Aprendí a utilizar consultas anidadas para combinar diferentes operaciones SQL, empleando el resultado de una consulta como criterio de selección de otra. Esto es importante porque permite resolver búsquedas más complejas, relacionar resultados calculados con registros específicos y automatizar comparaciones dinámicas sin establecer valores manualmente. Además, facilita la identificación de registros que cumplen condiciones basadas en información obtenida de la propia base de datos, mejorando la flexibilidad y precisión del análisis.
Aplicación real	En una empresa de comercio electrónico, esta técnica puede utilizarse para identificar el producto que generó el mayor ingreso por ventas y recuperar sus características comerciales. En una compañía de transporte, permite identificar al conductor que registró la mayor cantidad de kilómetros recorridos durante un periodo, facilitando la evaluación operativa. Asimismo, en una entidad financiera, puede emplearse para identificar las operaciones cuyo importe coincide con la transacción más alta registrada, recuperando sus detalles para una revisión o análisis de riesgos.

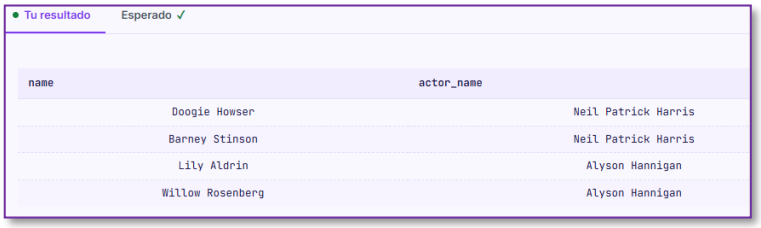
EJERCICIO 19 | Nombre: [NULL]

Objetivo	Aprender a identificar y filtrar registros que contienen valores nulos o no nulos mediante los operadores IS NULL e IS NOT NULL, permitiendo consultar información según la disponibilidad de datos en una tabla.
Concepto SQL	En SQL, NULL representa la ausencia de un valor o un dato desconocido. La condición IS NULL permite identificar registros que no tienen un valor definido en una columna, mientras que IS NOT NULL recupera aquellos que sí contienen un valor.
Consulta ejecutada	SELECT* FROM family_members WHERE favorite_book IS NOT NULL;
Resultado / captura	La consulta devolvió dos registros: Dave, cuyo libro favorito es To Kill a Mockingbird, y Mary, cuyo libro favorito es Gone with the Wind. Tu resultado Esperado ✓ id name gender species favorite_book 1 Dave male human To Kill a Mockingbird 2 Mary female human Gone with the Wind
Interpretación	La consulta recupera todas las columnas de la tabla family_members, pero únicamente muestra los registros cuyo campo favorite_book contiene un valor distinto de NULL. Como resultado, se obtienen dos filas con información registrada, excluyendo aquellas donde el campo no tiene un valor definido.
Aprendizaje	Aprendí a utilizar IS NULL e IS NOT NULL para identificar registros con información ausente o disponible dentro de una base de datos. Esto es importante porque permite evaluar la integridad y completitud de los datos, detectar campos pendientes de actualización y evitar interpretaciones incorrectas durante las consultas. Además, facilita los procesos de validación, depuración y control de calidad de la información, contribuyendo a mantener bases de datos más confiables para el análisis y la toma de decisiones.
Aplicación real	En una empresa de recursos humanos, esta instrucción puede utilizarse para identificar trabajadores que todavía no tienen registrada una fecha de finalización de contrato, diferenciándolos de aquellos con dicha información. Asimismo, en una organización que administra proyectos, puede emplearse para consultar actividades que ya tienen asignado un responsable o identificar aquellas que todavía no cuentan con uno, facilitando el control de tareas pendientes y la calidad de los registros.

EJERCICIO 20 | Nombre: [Fecha]

Objetivo	Aprender a realizar consultas SQL utilizando condiciones de comparación sobre campos de tipo fecha, permitiendo recuperar registros anteriores o posteriores a una fecha determinada.
Concepto SQL	SQL permite almacenar y comparar fechas mediante tipos de datos como DATE, que generalmente utiliza el formato YYYY-MM-DD (año, mes y día). Los operadores de comparación como >, <, >= y <= permiten filtrar registros según criterios temporales, facilitando búsquedas basadas en fechas específicas.
Consulta ejecutada	SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';
Resultado / captura	La consulta devolvió dos registros: Justin Timberlake, nacido el 31 de enero de 1981, y Taylor Swift, nacida el 13 de diciembre de 1989. Ambos cumplen la condición de haber nacido después del 1 de septiembre de 1980. Tu resultado Esperado ✓ id name birthdate 2 Justin Timberlake 1981-01-31 3 Taylor Swift 1989-12-13
Interpretación	La consulta recupera todas las columnas de la tabla celebs_born, pero únicamente muestra los registros cuya fecha de nacimiento (birthdate) es posterior al 1 de septiembre de 1980. El operador > permite realizar la comparación cronológica, excluyendo las fechas anteriores o iguales al límite establecido.
Aprendizaje	Aprendí a utilizar operadores de comparación sobre campos de tipo fecha para recuperar registros según criterios temporales específicos. Esto es importante porque permite realizar consultas cronológicas, identificar información correspondiente a determinados periodos y automatizar el seguimiento de fechas relevantes. Además, facilita el control de vencimientos, la evaluación de plazos y la elaboración de reportes históricos, contribuyendo a una gestión más eficiente y precisa de la información.
Aplicación real	En una empresa farmacéutica, esta instrucción puede utilizarse para identificar lotes de medicamentos cuya fecha de vencimiento sea posterior a una fecha determinada, facilitando el control de productos disponibles. En una entidad pública, permite consultar expedientes administrativos ingresados después de una fecha específica para realizar su seguimiento. Asimismo, en una empresa de servicios, puede emplearse para identificar contratos próximos a vencer o revisar mantenimientos realizados después de una fecha establecida, contribuyendo al cumplimiento de plazos y la planificación operativa.

EJERCICIO 21 | Nombre: [Inner joins]

Objetivo	Aprender a relacionar información almacenada en diferentes tablas mediante INNER JOIN, utilizando campos comunes para recuperar datos asociados en una sola consulta.
Concepto SQL	INNER JOIN permite combinar registros de dos tablas cuando existe una coincidencia entre los campos definidos mediante ON. Generalmente, esta relación se establece entre una clave primaria y una clave foránea, devolviendo únicamente las filas que cumplen la condición de unión.
Consulta ejecutada	SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;
Resultado / captura	La consulta devolvió cuatro registros que relacionan los personajes con sus respectivos actores. 
Interpretación	La consulta relaciona las tablas character y character_actor mediante los campos character.id y character_actor.character_id. Al encontrar coincidencias entre ambos identificadores, recupera el nombre de cada personaje junto con el actor que lo interpreta, integrando información de dos tablas diferentes.
Aprendizaje	Aprendí a utilizar INNER JOIN para relacionar información almacenada en diferentes tablas mediante campos comunes. La principal dificultad fue identificar qué columnas permitían establecer correctamente la relación, en este caso hice uso de las 3 pistas disponibles para el ejercicio y pude comprender la sintaxis correcta asociada a la información que se consultaba.
Aplicación real	En un hospital, esta instrucción puede utilizarse para relacionar una tabla de pacientes con otra de citas médicas, obteniendo el nombre del paciente junto con la especialidad y fecha de atención. Asimismo, en una organización, puede emplearse para unir las tablas de proyectos y asignaciones de personal, identificando los trabajadores que participan en cada proyecto sin duplicar su información en diferentes tablas.

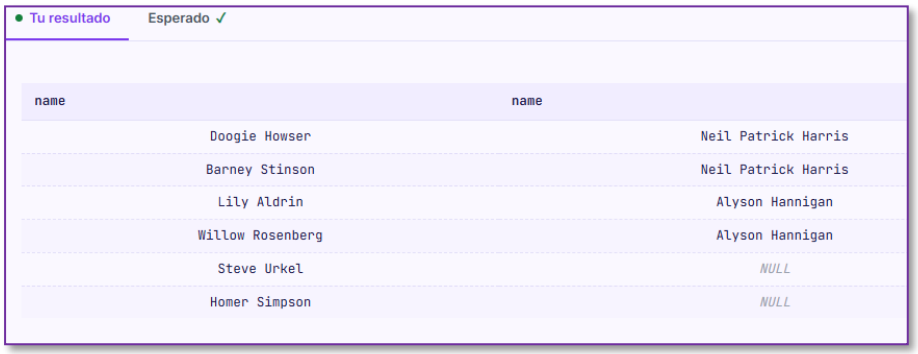
EJERCICIO 22 | Nombre: [Múltiples joins]

Objetivo	Aprender a utilizar múltiples instrucciones INNER JOIN para relacionar información almacenada en tres o más tablas mediante claves comunes, obteniendo datos relacionados en una sola consulta.
Concepto SQL	La instrucción INNER JOIN permite combinar registros de diferentes tablas mediante condiciones establecidas con ON. Cuando la información se encuentra distribuida en más de dos tablas, pueden utilizarse múltiples JOIN de manera consecutiva, relacionando claves primarias y foráneas para recuperar información sin duplicar datos innecesariamente.
Consulta ejecutada	SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;
Resultado / captura	La consulta devolvió cuatro registros que relacionan a los personajes con sus actores. 
Interpretación	La consulta relaciona tres tablas: character, character_actor y actor. El primer INNER JOIN vincula cada personaje con su identificador de actor mediante la tabla intermedia, mientras que el segundo permite obtener el nombre correspondiente desde actor. Finalmente, se muestran únicamente los nombres de los personajes y sus respectivos actores.
Aprendizaje	La principal dificultad fue comprender cómo estructurar la consulta e identificar qué tablas y columnas debían relacionarse en cada JOIN. Para resolverlo, analicé las claves primarias y foráneas, identificando que character_actor funciona como tabla intermedia entre character y actor.
Aplicación real	En una empresa de comercio electrónico, esta técnica puede usarse relacionar las tablas de pedidos, detalles de pedidos y productos para conocer qué artículos contiene cada compra. En una aerolínea, puede utilizarse para vincular pasajeros, reservas y vuelos, obteniendo la información del itinerario correspondiente a cada persona.

EJERCICIO 23 | Nombre: [Joins con WHERE]

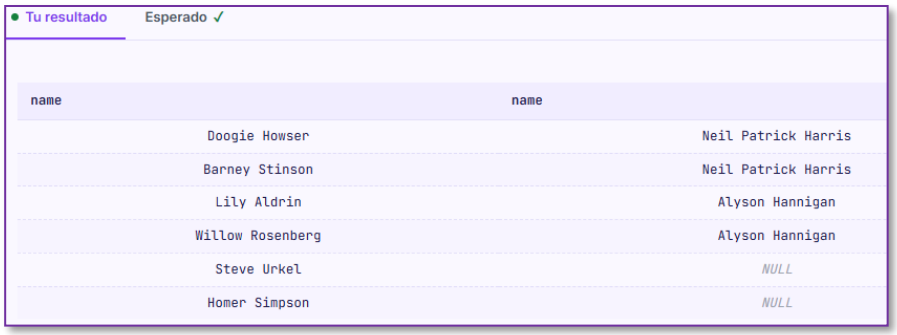
Objetivo	Aprender a combinar múltiples instrucciones INNER JOIN con la cláusula WHERE para relacionar información de diferentes tablas y recuperar únicamente los registros que cumplan condiciones específicas.
Concepto SQL	La instrucción INNER JOIN permite relacionar registros de diferentes tablas mediante campos comunes, mientras que WHERE permite filtrar los resultados obtenidos.
Consulta ejecutada	SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';
Resultado / captura	La consulta devolvió un único registro correspondiente al personaje Doogie Howser, asociado al programa Doogie Howser, M.D.. 
Interpretación	La consulta relaciona las tablas character, character_tv_show y tv_show mediante dos instrucciones INNER JOIN. Posteriormente, utiliza WHERE junto con el operador <> y AND para excluir al personaje Willow Rosenberg y los registros asociados al programa How I Met Your Mother, mostrando únicamente las coincidencias restantes.
Aprendizaje	Aprendí a combinar múltiples INNER JOIN con la cláusula WHERE para recuperar información relacionada y aplicar filtros específicos. El principal desafío fue comprender cómo integrar las condiciones después de las uniones sin confundir las columnas de cada tabla. Para resolverlo, identifiqué primero las relaciones mediante las claves primarias y foráneas y posteriormente apliqué los filtros con <> y AND.
Aplicación real	En una empresa hotelera, esta técnica puede utilizarse para relacionar huéspedes, reservas y habitaciones, excluyendo las reservas canceladas o habitaciones fuera de servicio. En una empresa de distribución, permite vincular proveedores, órdenes de compra y productos, excluyendo determinados proveedores o categorías de mercancía.

EJERCICIO 24 | Nombre: [Left joins]

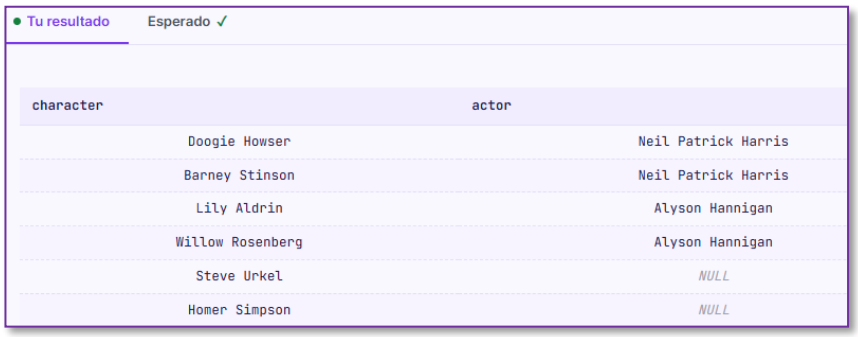
Objetivo	Aprender a utilizar LEFT JOIN para relacionar información de diferentes tablas, recuperando todos los registros de la tabla principal, incluso cuando no existan coincidencias en las tablas relacionadas.
Concepto SQL	LEFT JOIN permite combinar registros de dos o más tablas, conservando todas las filas de la tabla izquierda y agregando los datos coincidentes de la tabla derecha. Cuando no existe una relación, SQL devuelve valores NULL en las columnas de la tabla relacionada.
Consulta ejecutada	SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;
Resultado / captura	La consulta arrojó seis registros. Cuatro personajes tienen actores asociados y los personajes con valor NULL debido a que no tienen actores relacionados en la base de datos. 
Interpretación	La consulta utiliza dos LEFT JOIN para relacionar las tablas character, character_actor y actor. Primero identifica las relaciones entre personajes y actores, y posteriormente recupera sus nombres. A diferencia de INNER JOIN, conserva todos los personajes de la tabla principal, asignando NULL cuando no existe información del actor correspondiente.

Aprendizaje	Aprendí a utilizar LEFT JOIN para recuperar registros aunque no tengan información relacionada en otras tablas. La principal dificultad fue comprender cómo mantener todos los registros de la tabla principal al realizar múltiples uniones. Para resolverlo, analicé las relaciones entre las claves primarias y foráneas y comparé los resultados con INNER JOIN. Esto me permitió comprender la importancia de identificar información faltante sin excluir registros relevantes.
Aplicación real	En una empresa de mantenimiento industrial, esta instrucción permite obtener todos los equipos registrados junto con sus órdenes de mantenimiento, incluyendo aquellos que todavía no tienen intervenciones programadas. En una universidad, puede utilizarse para relacionar estudiantes con sus becas, mostrando también a quienes no tienen una asignada. Asimismo, en una empresa de desarrollo de software, permite consultar todos los proyectos junto con sus responsables, identificando aquellos que todavía no cuentan con un líder asignado y requieren seguimiento.

EJERCICIO 25 | Nombre: [Alias de tabla]

Objetivo	Aprender a utilizar alias para asignar nombres temporales a las tablas SQL, simplificando la escritura de consultas que relacionan múltiples tablas mediante instrucciones JOIN.
Concepto SQL	La palabra clave AS permite asignar un alias temporal a una tabla, facilitando su referencia dentro de una consulta. Los alias ayudan a reducir la longitud del código, identificar columnas de diferentes tablas y mejorar la legibilidad, especialmente cuando se utilizan múltiples instrucciones JOIN.
Consulta ejecutada	SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;
Resultado / captura	La consulta arrojó seis registros. Cuatro personajes tienen actores asociados y los personajes con valor NULL debido a que no tienen actores relacionados en la base de datos. 
Interpretación	La consulta utiliza los alias c, ca y a para representar las tablas character, character_actor y actor, respectivamente. Mediante dos instrucciones LEFT JOIN, relaciona los personajes con sus actores y conserva todos los registros de la tabla principal, incluso aquellos que no tienen coincidencias.
Aprendizaje	Aprendí a utilizar alias mediante AS para simplificar las consultas SQL que relacionan múltiples tablas. La principal dificultad fue comprender cómo reemplazar correctamente los nombres originales y mantener las referencias en cada JOIN. Para resolverlo, identifiqué las tres tablas involucradas y asigné sus respectivos alias, verificando las relaciones entre sus claves. Esto me permitió escribir consultas más cortas, legibles y fáciles de mantener, reduciendo la complejidad del código.
Aplicación real	En una empresa de telecomunicaciones, los alias pueden utilizarse para simplificar consultas que relacionan abonados, planes contratados y facturación, facilitando la elaboración de reportes comerciales. En una compañía de transporte, permiten escribir consultas más claras al relacionar vehículos, conductores y rutas asignadas. Asimismo, en un sistema de gestión documental corporativo, pueden emplearse para vincular documentos, versiones y usuarios responsables, facilitando la identificación de cada tabla y el mantenimiento de consultas complejas.

EJERCICIO 26 | Nombre: [Alias de columna]

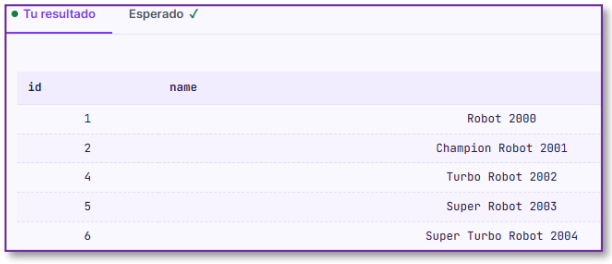
Objetivo	Aprender a utilizar alias para asignar nombres temporales a las columnas de una consulta SQL, facilitando la identificación y presentación de información obtenida de diferentes tablas.
Concepto SQL	La palabra clave AS permite asignar un nombre alternativo a una columna en el resultado de una consulta. Esto facilita la interpretación de la información, especialmente cuando diferentes tablas contienen campos con el mismo nombre. Los alias modifican únicamente los encabezados del resultado, sin alterar la estructura original de la base de datos.
Consulta ejecutada	SELECT character.name AS character,actor.name AS actor FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;
Resultado / captura	La consulta devolvió seis registros organizados en dos columnas denominadas character y actor. 
Interpretación	La consulta relaciona las tablas character, character_actor y actor mediante dos instrucciones LEFT JOIN, conservando todos los personajes, incluso aquellos sin actores asociados. Posteriormente, utiliza AS para renombrar las columnas del resultado como character y actor, permitiendo distinguir claramente los nombres de los personajes y sus respectivos intérpretes.
Aprendizaje	Aprendí a utilizar alias de columna mediante AS para presentar los resultados de una consulta de manera más clara y organizada. La principal dificultad fue diferenciar los alias de tabla de los alias de columna, ya que ambos utilizan la misma instrucción. Para resolverlo, analicé su ubicación dentro de la consulta y comprobé que los alias de columna modifican los encabezados del resultado, mientras que los de tabla simplifican las referencias en el código. Esto facilita la interpretación de consultas complejas y evita confusiones entre campos con nombres similares.
Aplicación real	En una empresa de comercio electrónico, los alias de columna pueden utilizarse para presentar campos técnicos como order_date y total_amount con encabezados más comprensibles, como «Fecha de pedido» e «Importe total». En una institución financiera, permiten diferenciar las columnas correspondientes al nombre del titular y del beneficiario de una transferencia.

character	actor
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	NULL
Homer Simpson	NULL

EJERCICIO 27 | Nombre: [Self joins]

Objetivo	Aprender a utilizar SELF JOIN para relacionar registros dentro de una misma tabla, mediante alias que permitan identificar diferentes roles y recuperar información jerárquica.
Concepto SQL	SELF JOIN es una técnica que permite relacionar una tabla consigo misma mediante una instrucción JOIN. Para ello, se utilizan alias que permiten diferenciar las dos referencias a la misma tabla. Es especialmente útil para consultar relaciones jerárquicas, como empleados y supervisores, utilizando claves primarias y foráneas.
Consulta ejecutada	SELECT e.name AS employee_name,b.name AS boss_name FROM employees AS e INNER JOIN employees AS b ON e.boss_id = b.id;
Resultado / captura	La consulta devolvió tres registros que relacionan a cada empleado con su jefe. 
Interpretación	La consulta utiliza dos referencias de la tabla employees, identificadas mediante los alias e (empleado) y b (jefe). A través de INNER JOIN, relaciona el campo e.boss_id con b.id, permitiendo obtener el nombre del empleado junto con el de su superior. Solo muestra los registros que tienen una relación válida entre ambos campos.
Aprendizaje	Aprendí a utilizar SELF JOIN para establecer relaciones entre registros de una misma tabla. La principal dificultad fue comprender cómo una tabla podía relacionarse consigo misma y por qué era necesario utilizar dos alias diferentes. Para resolverlo, analicé el orden lógico de la consulta, identificando que FROM y JOIN establecen las referencias antes de seleccionar las columnas, y relacioné boss_id con id. Esto me permitió comprender cómo consultar estructuras jerárquicas sin duplicar tablas.
Aplicación real	En una empresa, esta técnica permite consultar la estructura organizacional y relacionar a cada trabajador con su jefe inmediato, facilitando la elaboración de organigramas y reportes de supervisión. En una empresa de distribución, puede utilizarse para relacionar sucursales regionales con sus sedes principales cuando ambas se encuentran registradas en una misma tabla.

EJERCICIO 28 | Nombre: [LIKE]

Objetivo	Aprender a utilizar el operador LIKE para buscar registros que coincidan con determinados patrones de texto, empleando caracteres comodín para establecer criterios de búsqueda flexibles.
Concepto SQL	El operador LIKE, utilizado con la cláusula WHERE, permite buscar coincidencias parciales en campos de texto. Utiliza dos caracteres comodín: %, que representa cero o más caracteres, y _, que representa exactamente un carácter. Esto permite identificar registros que contienen, comienzan o terminan con determinados patrones.
Consulta ejecutada	SELECT * FROM robots WHERE name LIKE '%Robot 20__';
Resultado / captura	La consulta devolvió cinco registros: Robot 2000, Champion Robot 2001, Turbo Robot 2002, Super Robot 2003 y Super Turbo Robot 2004. Todos contienen la palabra Robot seguida de un año entre 2000 y 2099, cumpliendo el patrón establecido. 
Interpretación	La consulta recupera los registros de la tabla robots cuyo campo name coincide con el patrón '%Robot 20__'. El símbolo % permite cualquier texto antes de Robot, mientras que los dos guiones bajos representan los dos caracteres posteriores a 20. De esta manera, se obtienen cinco coincidencias y se excluyen los nombres que no cumplen el patrón.
Aprendizaje	Aprendí a utilizar LIKE junto con los comodines % y _ para realizar búsquedas por patrones de texto. El principal desafío fue comprender la diferencia entre ambos símbolos y cómo combinarlos para obtener coincidencias específicas. Para resolverlo, analicé la estructura de los nombres y utilicé % para permitir texto inicial variable y _ para limitar los caracteres finales. Esto permite realizar búsquedas flexibles y localizar información sin conocer el valor completo de un campo.
Aplicación real	En una empresa de manufactura, esta instrucción puede utilizarse para localizar productos cuyos códigos comiencen con un prefijo correspondiente a una línea de producción, facilitando su clasificación. En una compañía de telecomunicaciones, permite buscar números de serie o identificadores de dispositivos que compartan un patrón de fabricación.

EJERCICIO 29 | Nombre: [CASE]

Objetivo	Aprender a utilizar la expresión CASE para evaluar diferentes condiciones y asignar valores específicos a los registros según los criterios establecidos.
Concepto SQL	La expresión CASE permite realizar evaluaciones condicionales dentro de una consulta SQL. Utiliza WHEN para establecer condiciones, THEN para definir el valor devuelto cuando se cumple una condición y ELSE para los demás casos. Finaliza con END y puede utilizar AS para asignar un nombre a la columna calculada.
Consulta ejecutada	SELECT *, CASE WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' ELSE 'talk' END AS sound FROM friends_of_pickles;
Resultado / captura	La consulta devolvió los siete registros originales y agregó una columna denominada sound. Se asignó talk a los dos humanos (Dave y Mary), meow a los dos gatos (Fry y Leela) y bark a los tres perros (Odie, Jumpy y Sneakers). Tu resultado Esperado ✓ id | name | gender | species | height_cm | sound 1 | Dave | male | human | 180 | talk 2 | Mary | female | human | 160 | talk 3 | Fry | male | cat | 30 | meow 4 | Leela | female | cat | 25 | meow 5 | Odie | male | dog | 40 | bark 6 | Jumpy | male | dog | 35 | bark 7 | Sneakers | male | dog | 55 | bark
Interpretación	La consulta evalúa el valor de species en cada registro mediante CASE. Cuando corresponde a dog, devuelve bark; cuando es cat, devuelve meow; y para los demás valores devuelve talk. Los resultados aparecen en una nueva columna denominada sound, sin modificar los datos originales de la tabla.
Aprendizaje	Aprendí a utilizar CASE para evaluar múltiples condiciones y devolver diferentes valores dentro de una consulta SQL. El principal desafío fue comprender el orden de evaluación de WHEN, THEN y ELSE. Para resolverlo, analicé los posibles valores de la columna species y asigné una respuesta a cada condición. Esto permite automatizar clasificaciones, aplicar reglas de negocio y presentar información de manera más comprensible sin modificar los datos originales.
Aplicación real	En una entidad financiera, esta expresión puede utilizarse para clasificar operaciones según su importe y asignarles niveles de revisión como «Normal», «Elevado» o «Crítico». En una empresa de atención al cliente, permite categorizar solicitudes según sus días de retraso como «En plazo», «Próxima a vencer» o «Vencida».

EJERCICIO 30 | Nombre: [SUBSTR]

Objetivo	Aprender a utilizar la función SUBSTR() para extraer partes específicas de una cadena de texto y aplicar filtros que permitan identificar registros según determinados caracteres.
Concepto SQL	La función SUBSTR() permite extraer una porción de texto indicando la posición inicial y, opcionalmente, la cantidad de caracteres. Admite posiciones negativas para contar desde el final de la cadena. Puede combinarse con WHERE y LIKE para filtrar registros según fragmentos específicos de su contenido.
Consulta ejecutada	SELECT * FROM robots WHERE SUBSTR(location, -2) LIKE 'NY';
Resultado / captura	La consulta devolvió cinco registros correspondientes a los robots ubicados en el estado de Nueva York (NY). Tu resultado Esperado ✓ id | name | location 1 | R2000 - Robot 2000 | New City, NY 3 | D0001 - Dragon | New York City, NY 4 | R2002 - Turbo Robot 2002 | Spring Valley, NY 5 | R2003 - Super Robot 2003 | Nyack, NY 8 | U2111 - Unreleased Turbo Robot 2111 | Buffalo, NY
Interpretación	La consulta utiliza SUBSTR(location, -2) para extraer los últimos dos caracteres del campo location de cada registro. Posteriormente, WHERE y LIKE 'NY' permiten seleccionar únicamente aquellos cuya ubicación termina en NY, excluyendo los registros correspondientes a otros estados.
Aprendizaje	Aprendí a utilizar SUBSTR() para extraer fragmentos específicos de texto y combinarlos con condiciones de búsqueda. Esto es importante porque permite procesar información estructurada dentro de campos de texto, realizar búsquedas precisas y automatizar la extracción de datos sin modificar los registros originales.
Aplicación real	En una empresa de mensajería, esta función puede utilizarse para extraer códigos de zona incluidos al final de los identificadores de envío y clasificar paquetes según su destino. En una compañía automotriz, permite obtener segmentos específicos de códigos de fabricación para identificar modelos o plantas de producción. Asimismo, en una organización que administra documentos electrónicos, puede emplearse para extraer prefijos o sufijos de códigos documentales que identifican el área responsable, el tipo de documento o su periodo de emisión, facilitando la clasificación y organización de información.

EJERCICIO 31 | Nombre: [COALESCE]

Objetivo	Aprender a utilizar la función COALESCE() para seleccionar el primer valor no nulo entre varias columnas, estableciendo un orden de prioridad para recuperar información disponible.
Concepto SQL	La función COALESCE() evalúa una lista de expresiones de izquierda a derecha y devuelve el primer valor diferente de NULL. Si todos los valores son nulos, devuelve NULL. Permite establecer prioridades entre campos y gestionar información incompleta sin modificar los datos originales.
Consulta ejecutada	SELECT name, COALESCE(tank, gun, sword) as weapon FROM fighters;
Resultado / captura	La consulta devolvió cuatro registros. Para US Marine seleccionó M1A1 Abrams Tank, para John Wilkes Booth .44 caliber Derringer y para Zorro Sword of Zorro. En el caso de Innocent Bystander, devolvió NULL porque no tiene ningún valor registrado en las tres columnas evaluadas. 
Interpretación	La consulta evalúa las columnas tank, gun y sword en ese orden, priorizando el tanque sobre el arma de fuego y la espada. COALESCE() devuelve el primer valor disponible para cada luchador y lo presenta en una columna denominada weapon. Si las tres columnas contienen NULL, el resultado también será nulo.
Aprendizaje	Aprendí a utilizar COALESCE() para recuperar el primer valor disponible entre varias columnas y establecer prioridades en una consulta SQL. El principal desafío fue comprender cómo el orden de los argumentos determina el resultado. Para resolverlo, analicé los valores nulos de cada registro y organicé las columnas según la prioridad solicitada. Esto es importante porque permite gestionar datos incompletos, establecer alternativas de información y simplificar las consultas sin recurrir a múltiples condiciones.
Aplicación real	En una empresa de logística, esta función puede utilizarse para seleccionar la dirección de entrega prioritaria de un pedido, utilizando una dirección alternativa cuando la principal no esté registrada. En una plataforma de atención al cliente, permite recuperar el medio de contacto preferido, como teléfono móvil, teléfono fijo o correo electrónico, según su disponibilidad. Asimismo, en una empresa de recursos humanos, puede emplearse para identificar el contacto de emergencia principal de un trabajador y utilizar uno secundario cuando el primero no esté registrado, facilitando la gestión de información incompleta.

Reflexión general

Durante el desarrollo de los 31 ejercicios de SQL Easy, pude fortalecer mis conocimientos sobre el lenguaje SQL, comenzando con consultas básicas y avanzando progresivamente hacia operaciones de mayor complejidad. Los temas que me resultaron más sencillos fueron las instrucciones SELECT, los filtros mediante WHERE, los operadores lógicos AND y OR, y las funciones de agregación como COUNT, SUM y AVG, debido a que su funcionamiento era más intuitivo y sus resultados fáciles de interpretar. Por otro lado, los ejercicios que representaron un mayor desafío fueron los relacionados con INNER JOIN, múltiples JOIN y SELF JOIN, principalmente por la dificultad para identificar las relaciones entre tablas, comprender el uso de claves primarias y foráneas, y determinar qué campos debían utilizarse en cada condición ON. Para superar estas dificultades, fue necesario analizar cuidadosamente la estructura de las tablas, identificar sus campos comunes y comprender el orden lógico de procesamiento de las consultas, lo que me permitió desarrollar una mejor comprensión del funcionamiento de las bases de datos relacionales.

Considero que los conocimientos adquiridos tienen una aplicación directa en mi entorno profesional, especialmente en las áreas de infraestructura tecnológica y ciberseguridad, donde el análisis de información es fundamental para la gestión y protección de los sistemas. Mediante SQL podría consultar registros de auditoría, analizar eventos de autenticación, identificar patrones de actividad sospechosa y generar reportes sobre incidentes de seguridad. Asimismo, el uso de instrucciones como WHERE, GROUP BY y JOIN permitiría filtrar, agrupar y relacionar información de usuarios, activos tecnológicos, sistemas y permisos de acceso, facilitando las revisiones de seguridad, la evaluación del cumplimiento de controles y la elaboración de indicadores para apoyar la toma de decisiones.

Finalmente, esta experiencia no solo me permitió mejorar mis habilidades para construir, ejecutar e interpretar consultas SQL, sino también fortalecer mi razonamiento lógico, capacidad de análisis y resolución de problemas. La práctica constante me ayudó a comprender la importancia de estructurar correctamente las consultas, interpretar sus resultados y documentar técnicamente cada procedimiento. Además, identificar aplicaciones reales para los conceptos aprendidos me permitió reconocer el valor de SQL como una herramienta para transformar datos en información útil, optimizar procesos y contribuir a una gestión más eficiente y segura de la información en las organizaciones.

Conclusión final

El desarrollo de los 31 ejercicios de SQL Easy me permitió fortalecer mis conocimientos sobre el lenguaje SQL y comprender su importancia para la consulta, organización y análisis de información almacenada en bases de datos relacionales. A través de una práctica progresiva, aprendí a utilizar instrucciones básicas y avanzadas, identificar relaciones entre tablas y aplicar diferentes criterios para obtener información precisa.

Uno de los aprendizajes más importantes fue comprender que SQL no consiste únicamente en recuperar datos, sino también en relacionarlos, interpretarlos y transformarlos en información útil para la toma de decisiones. Las dificultades encontradas en los ejercicios de JOIN contribuyeron especialmente al desarrollo de mi razonamiento lógico y a una mejor comprensión de la estructura de las bases de datos.

Finalmente, considero que los conocimientos adquiridos tienen una aplicación directa en mi desarrollo profesional, especialmente en áreas de infraestructura tecnológica, auditoría y ciberseguridad, donde el análisis de datos resulta fundamental para el monitoreo de sistemas, la identificación de riesgos y la mejora de los controles de seguridad. Esta experiencia representa una base importante para continuar profundizando en SQL y su aplicación en escenarios empresariales de mayor complejidad.